Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Why searching matters

Today
Welcome to your next lesson: Why searching matters.

I'm getting your personalized lesson ready — based on what you've learned so far, what's next, and the best way for you to practice.

Step by step, you’re moving forward.

Ron, let’s start simple 🍸

Imagine you have 800 contacts on your phone, and you want to call “Alex”.

You could:

Scroll from the top through every contact until you see Alex, or
Tap the search bar, type “Al…”, and jump straight to Alex.

Both are ways to find Alex, but one clearly feels faster and less annoying.

I want to hear from you 👇

Name one place in your own life where you often have to find something:

For example: contacts, emails, files, messages, game items, songs, etc.

Just give me one concrete example from your day-to-day.

look up a specific trades information on my auto stocks and crypto tradig bot program (pythong with sqlite db)

Nice, that’s a perfect example 👌

So your situation is:

You have many trades stored in an SQLite database (via Python).
You want to look up one specific trade’s information (maybe by ID, symbol, date, etc.).

In programming terms, that’s exactly a search task:

You have a collection (all trades in your DB) and a goal (find the one trade that matches some condition).

Quick check for you, just to lock this in:

When you “look up a specific trade” in your program, what is being searched?

The SQLite table rows
The Python source code files

Just answer in your own words (no need to be fancy) — which one are you actually searching through?

yjr sqlite table rows

Perfect, and I see the typo — you meant the SQLite table rows 👍

So in your example:

What is being searched? Rows in a trades table (each row = one trade)
Where is it stored? In an SQLite database, accessed from Python

That’s exactly how I want you to think about search tasks:

“What am I looking for?” and “Where is the collection it lives in?” 🔍

Next step: let’s look at why how you search those rows can really affect speed, especially as your number of trades grows.

Before I go there, does this mapping make sense?

“Find specific trade” → search task
“Trades table rows” → collection being searched

Yes / no?

Yes, that makes sense

What do you think? 👇

Which of these is definitely a search task?

A.

Looping over all rows in your trades table to find a trade with a specific trade_id.

B.

Calculating profit/loss for a single trade when you already have that trade row in a variable.

C.

Saving a new trade into the database.

Nice pick 🎯 You nailed it.

Looping over all rows to find a specific trade_id is definitely a search task because:
You have a collection: all rows in the trades table.
You have a target: the row whose trade_id matches what you want.
Your code is literally "looking through" items to find a match.

The other ones aren't really searches:

Calculating profit/loss for a trade: you already have the trade; now you're just doing math on it.
Saving a new trade: that's inserting data, not looking for existing data.
Let's build on this using your trading bot example.

Imagine two approaches:

You pull all trades into Python and loop over them to find one ID.
You use a direct lookup, like a dictionary in Python or an indexed SELECT in SQL, that jumps straight to the trade.

Both work, but they don't cost the same in time.

I'll explain the idea of full scan vs direct lookup next, step by step.

Before I do: do you want me to relate it more to Python lists vs dicts, or more to SQL queries and indexes?

Python lists vs dicts

Great, Python lists vs dicts is a perfect lens for this 🔍

Very high level:

A list search usually means:
"Start at index 0, check each element one by one until I find what I want."
→ Time grows with how many items are in the list.

A dict lookup usually means:
"I already know the key; jump straight to the value."
→ Time is almost the same whether you have 10 items or 1,000,000 (in practice).

So:

List search = full scan (O(n))

Dict lookup = direct access (O(1) in the normal case)

We won't dive into internals, just the effect on speed.

To be sure this is clear, let me ask you this in words:

If you have 1,000,000 trades in a Python list, and you search by looping, what happens to how long it takes as you add more trades — does it stay about the same, or does it grow with the number of trades?

It grows with the list size

Exactly 💯

With a list, if you double the number of items, your worst□case search time roughly doubles too. You might have to walk through almost every element.

Later we'll give that the label O(n), but for now:

More items in the list ⇒ more work when you search.

Next step, I want to contrast that with a dict.

At a high level:

With a dict, when you do trades[trade_id], Python jumps straight to the value using the key.
The time for that lookup stays almost the same, even as the dict gets huge (ignoring some edge cases).

Make sense so far:

list search → grows with list size
dict lookup → roughly constant time?

If yes, I'll tie this back directly to why "choosing a good search approach" matters for real programs like your trading bot.

Yes, that makes sense

Awesome. Let's connect this to why it matters for real programs like your bot ❁

Imagine two versions of your Python code:

List-based search

You load trades into a list: trades_list
To find a trade, you loop: for trade in trades_list: ...
More trades ⇒ more looping ⇒ slower as history grows
Dict-based lookup

You store trades in a dict keyed by trade_id: trades_by_id[trade_id] = trade
To find a trade, you just do: trade = trades_by_id[trade_id]
Whether you have 1,000 or 1,000,000 trades, lookup time is almost the same

So in practice:

Same goal: "find this trade"
Different approach: list scan vs dict lookup
Different user experience: fast now vs slow and laggy later

That's the core idea of this lesson:

Choosing a good search approach can make your program stay fast as data grows ❁

To check you've got the idea, not the jargon:

If your trading bot keeps growing its trade history, which structure is usually better

for fast “find by trade_id” in Python — a plain list or a dict keyed by trade_id? Just answer in one word: list or dict.

a dict and its what i use

Love that — you’re already making the “good search” choice in your own code 🔥

So far you’ve shown you can:

Spot a search task (finding a specific trade in the DB)
Say what is being searched (SQLite rows / trades collection)
Explain why dict vs list matters for speed as data grows
Next, I want to zoom out from Python a bit and practice recognizing search tasks in everyday situations (not just code). That’s key for this lesson.

Here’s a quick one for you:

You open your email app and want to find one specific message from your broker about a margin call. You type the broker’s name into the search bar.

Would you call that a search task in our sense?

Just answer: yes or no, and if you want, say what is being searched.

yes

Yes ☑

In that email example:

What’s being searched? All your emails (the collection)
What are you looking for? A specific message from your broker

So it fits our pattern: a search task = find one item inside a bigger collection.

Let’s practice spotting search tasks in a few different situations to really lock this in 🔒

For each of these, tell me which ones you think are searches (you can just reply with the letters):

A. Scanning your downloads folder to find report.pdf

B. Compressing a big video file to make it smaller

C. Typing a friend’s username into a game lobby to invite them

D. Sorting a list of numbers from smallest to largest

Which of A, B, C, D are search tasks?

searches: A, D (possibly C if you conisder the server had to search all the users for whatyou type)

Nice reasoning, and I like that you’re thinking about what the server does under the hood 🧠

Let’s go one by one:

A. Scanning your downloads folder to find report.pdf → Yes, a search

Collection: all files in Downloads
Target: the file named report.pdf
C. Typing a friend’s username into a game lobby → Also a search (like you hinted)

Collection: all players / accounts the server knows about
Target: the player whose username matches what you typed
D. Sorting a list of numbers → Not a search in our sense

You’re not looking for one specific item; you’re rearranging all items into order.

So the real “search tasks” here are A and C ✔

D is important in algorithms, but it’s a different kind of problem.

Quick check-in before we do one more short exercise:

Does this rule feel clear?

A task is a search when the main goal is “find this specific thing inside a collection,” not “change/reorder all the things.”

Yes / no?

teu i was rasoning you need to search through at eaach stage isert or not but your right

Yeah, your reasoning about D makes sense — during sorting, the algorithm is kind of “looking around” to decide where to insert things. But the overall goal is different:

Sorting: “Put all items into order.”
Searching: “Find this specific item inside the collection.”

You’re thinking at a good depth there 👌

Let’s do one more quick classification to really cement this and then we’ll move back to code-land.

Consider these:

Your bot loads the last 10,000 trades and finds the largest profit trade.
Your bot checks whether there is any trade with status = 'FAILED'.
Your bot takes one known trade and updates its fee field.

Which of these are search tasks?

You can answer with numbers (like: 1 and 2 or only 2, etc.).

1 and 2 even with dicts you need to search them all for those fields

Exactly 💡

1. Find the largest profit trade → search

You have a collection (10,000 trades).
You’re looking for a specific one: “the max-profit trade.”
Even if you scan all of them, the goal is “find that one.”
2. Check whether there is any trade with status = 'FAILED' → also search

Collection: all trades.
Goal: “Is there at least one that matches this condition?”
Under the hood, that’s still “look through trades until I find one (or confirm none).”
3. Update fee on one known trade → not a search

You already know exactly which trade you’re updating (you’ve got the row/object).
Now you’re just modifying data, not looking for it.

You’re also right that for some conditions (like “largest profit”), even with dicts you still need to go through many entries — and that’s where the cost can really add up.

You’ve basically hit all the key points for this lesson:

Spotting search tasks in real life and in your bot
Saying what’s being searched and what the target is
Feeling why different approaches (full scan vs direct lookup) matter for speed

Want to do one quick multiple-choice “victory lap” question to lock it in, or would you rather move on to the next lesson after this?

Victory lap question

What do you think? Take your time and pick the best answer 👇

Which scenario is definitely a search task in our sense?

A.

Looping over all rows in your trades table to find a trade with a specific trade_id.

B.

Calculating profit/loss for a single trade when you already have that trade row in a variable.

C.

Saving a new trade into the database.

Nice finish, Ron 🎯

You picked the right one:

Looping over all rows in your trades table to find a trade with a specific trade_id is a pure search task:
Collection: all rows in the table
Goal: find the one row with that trade_id
The whole job is “look through data to find a match.”

The others are different kinds of work:

Calculating P/L: you already have the trade, you’re just doing math.
Saving a new trade: that’s writing data, not looking for existing data.
From what you’ve shown, you’ve got the key outcomes for this lesson:

You can spot search tasks (in your trading bot, email, files, games).
You understand what is being searched and where it lives.
You get why list scan vs dict lookup matters for speed as data grows.

Are you ready to wrap up this lesson and move on to the next one on searching and sorting, or do you want to poke at one more example first?

Move to the next lesson

Nice work, Ron 🎉

Quick recap of what you just nailed:

You can spot search tasks in real life and in code (emails, files, game lobbies, trading bot, DB rows).
You can say what's being searched (the collection) and what you're looking for (the target).
You understand that:
Full scans (like looping a list or table) get slower as data grows.
Direct lookups (like Python dicts keyed by trade_id) stay fast even as things scale.
That's exactly the foundation you need before we dive into actual search algorithms next 👊

When you're ready, hit Next Lesson in the Learn tab and we'll start getting into how to implement these searches step by step in Python.